

PROJECT REPORT

AI/ML-Based Predictive Maintenance and Vehicle Health Monitoring System

Module 3 (Connectivity) Project — Project 1

Student Name	Abhishek Ghosh
Roll / Registration No.	07
Department	Computer Science / AI & ML
Semester	7
Institution	Institute of Engineering and Management, New Town
Academic Year	2023–2027

ABSTRACT

Predictive maintenance aims to identify abnormal vehicle conditions before a component failure occurs. This project develops an AI/ML-based vehicle health monitoring system that uses operational parameters such as engine temperature, RPM, oil pressure, vibration, battery voltage, coolant temperature, fuel consumption, vehicle speed, and operating hours. The workflow performs data preprocessing, train-test splitting, feature scaling, and classification using a Random Forest model. The trained model predicts failure risk and converts the prediction probability into a vehicle health score. Based on the health score, the system reports NORMAL, WARNING, or CRITICAL status and generates a maintenance alert. Feature-importance analysis is also used to identify the vehicle parameters that contribute most to the model's prediction.

1. INTRODUCTION

Vehicle failures can reduce safety, increase maintenance costs, and cause unexpected downtime. Traditional maintenance often depends on fixed service intervals or inspection after a problem becomes visible. A predictive maintenance system instead uses operating data to estimate the possibility of failure and provide an early warning.

The Module 3 specification defines the goal of this project as predicting vehicle or component failure before it occurs. The proposed architecture follows the sequence: vehicle sensors, data preprocessing, ML/DL model, failure probability, vehicle health score, and maintenance alert.

2. PROJECT OBJECTIVES

1. Collect and organize vehicle operating parameters for health monitoring.
2. Preprocess the input data so it can be used by a machine-learning model.
3. Train a Random Forest classifier to identify failure risk.
4. Estimate failure probability for test samples.
5. Convert failure probability into a vehicle health score.
6. Classify vehicle condition as NORMAL, WARNING, or CRITICAL.
7. Generate a maintenance alert based on the predicted condition.
8. Analyze feature importance to identify influential vehicle parameters.
9. Visualize health score, failure probability, and status distribution.

3. SYSTEM REQUIREMENTS

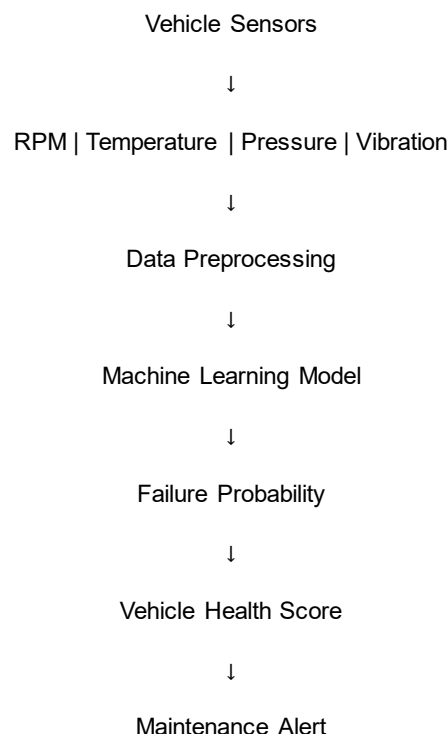
Category	Requirement
Programming Language	Python
Environment	Google Colab / Jupyter Notebook
Data Format	CSV

Main ML Model	Random Forest Classifier
Libraries	Pandas, NumPy, Scikit-learn, Matplotlib, Seaborn
Target	Failure (0 = No Failure, 1 = Failure)

4. INPUT PARAMETERS

No.	Parameter	Purpose
1	Engine Temperature	Represents engine thermal condition.
2	RPM	Represents engine rotational speed.
3	Oil Pressure	Indicates lubrication-system condition.
4	Vibration	Represents mechanical vibration level.
5	Battery Voltage	Indicates electrical/battery condition.
6	Coolant Temperature	Represents cooling-system condition.
7	Fuel Consumption	Represents fuel usage during operation.
8	Vehicle Speed	Represents operating speed.
9	Operating Hours	Represents accumulated operating time.

5. SYSTEM ARCHITECTURE



6. DATASET

For the implemented prototype, a CSV dataset named **vehicle_health_dataset.csv** was created for the nine specified vehicle parameters and a binary target column named **Failure**. The prototype dataset contains 1,000 samples. The Failure target uses 0 for no failure and 1 for failure/risk. The dataset was used to demonstrate the complete machine learning pipeline in Google Colab.

Important: this prototype dataset is suitable for demonstrating the project workflow, but it should not be presented as real-world vehicle sensor data. For real deployment, the model should be trained and validated using authentic vehicle telemetry and properly labelled failure events.

7. DATA PREPROCESSING

1. The CSV dataset was loaded using Pandas.
2. The Failure column was separated as the target variable y .
3. The remaining nine parameters were used as input features X .
4. The data was divided into training and testing sets using an 80:20 split.
5. StandardScaler was applied to scale the input features.

8. RANDOM FOREST MODEL

Random Forest Classifier was selected as the main model for the implemented prototype. It combines multiple decision trees and uses their combined predictions to perform classification. In this project, the model receives the preprocessed vehicle parameters and predicts whether a failure condition is likely.

The implemented configuration uses 100 decision trees ($n_estimators = 100$) with a fixed random state of 42 for reproducibility. The model is trained on the training portion of the dataset and evaluated on the held-out test data.

9. FAILURE PROBABILITY

After training, the model's `predict_proba()` function is used to obtain the estimated probability of the Failure = 1 class. The probability is converted into a percentage for easier interpretation.

Failure Probability (%) = Predicted Failure Probability $\times$ 100

10. VEHICLE HEALTH SCORE

For this prototype, the vehicle health score is defined as the complement of the failure probability:

Vehicle Health (%) = 100 – Failure Probability (%)

11. STATUS CLASSIFICATION

Health Score	Status	Maintenance Alert
70% or above	NORMAL	No maintenance required
40% to below 70%	WARNING	Schedule maintenance soon
Below 40%	CRITICAL	Immediate maintenance required

12. MODEL EVALUATION

The model was evaluated using accuracy, classification report, and confusion matrix. The classification report provides precision, recall, and F1-score. The confusion matrix compares actual and predicted classes and helps identify correct predictions, false alarms, and missed failures.

Actual Accuracy from Colab: _____ %

The blank above should be replaced with the exact percentage displayed by the completed Colab notebook before final submission.

13. FEATURE IMPORTANCE

Random Forest provides `feature_importances_` values that indicate the relative importance of the input features for the trained model. The project displays these values in descending order and visualizes them using a horizontal bar chart. The top five features from the actual Colab output should be inserted here after the final run.

Rank	Feature	Importance
1	_____	_____
2	_____	_____

3	_____	_____
4	_____	_____
5	_____	_____

14. SAMPLE FINAL OUTPUT

Output	Example
Vehicle Health	72%
Failure Probability	68%
Status	WARNING
Major Contributing Factors	High vibration; Rising temperature; Abnormal oil pressure

The above values are the sample output specified in the Module 3 project document, not a claim about the prototype's actual prediction for a particular vehicle.

15. VISUALIZATION

1. Vehicle Health Score graph for the first 30 test samples.
2. Failure Probability graph for the first 30 test samples.
3. Vehicle Status Distribution showing NORMAL, WARNING, and CRITICAL counts.
4. Feature Importance graph showing the relative importance of vehicle parameters.

16. ADVANTAGES

- Provides an automated vehicle health monitoring workflow.
- Produces a failure probability rather than only a binary prediction.
- Converts model output into an easy-to-understand health score.
- Generates maintenance alerts based on predefined health thresholds.
- Provides feature-importance information for basic model interpretation.
- Can be extended with additional vehicle sensors and real-time telemetry.

17. LIMITATIONS

- The prototype dataset is generated for project demonstration and is not a substitute for real vehicle telemetry.
- The current prototype is not connected to live vehicle sensors.
- The health-score thresholds are simple project rules and should be validated for real deployment.
- Failure labels and real-world failure patterns require domain-specific validation.
- The model's performance may change when trained on authentic data from different vehicles or operating environments.

18. FUTURE SCOPE

- Connect the system to real-time vehicle sensors.
- Use authentic and larger vehicle maintenance datasets.
- Compare Random Forest with XGBoost and other suitable models.
- Explore time-series models such as LSTM or GRU for sequential sensor data.
- Add explainable AI methods for more detailed prediction explanations.
- Develop a web or mobile dashboard for real-time monitoring.
- Add notification services for maintenance alerts.

19. CONCLUSION

This project demonstrates an end-to-end AI/ML workflow for predictive maintenance and vehicle health monitoring. Vehicle operating parameters are preprocessed and supplied to a Random Forest classifier. The model output is then transformed into failure probability and vehicle health score, followed by a simple status and maintenance-alert mechanism. Visualization and feature importance provide additional understanding of the model output. The prototype fulfills the main workflow described in the Module 3 project specification and provides a foundation that can later be extended to real-time vehicle data.

20. REFERENCES

1. Module 3 (Connectivity) Project specification provided by the course/institution.
2. Scikit-learn documentation — RandomForestClassifier, train_test_split, StandardScaler, and evaluation metrics.
3. Pandas documentation — CSV data handling and DataFrame operations.
4. Matplotlib and Seaborn documentation — data visualization.

Submission note: Before submitting, add your name, roll number, institution details, the actual Colab accuracy, and screenshots of your dataset, confusion matrix, feature-importance graph, health graph, and final vehicle report.